Note Processing System

Tarun Boddeda
Blekinge Institute of Technology
Karlskrona, Sweden
Email: tabo25@student.bth.se

Ruthik Garapati
Blekinge Institute of Technology
Karlskrona, Sweden
Email: ruga25@student.bth.se

***Abstract*—Unstructured digital note-taking often results in chaotic information management where tasks, questions, and time-sensitive reminders are buried within text. This project addresses the challenge of automatically classifying raw notes into three distinct categories: Questions, Tasks, and Deadlines/Reminders. Unlike modern approaches relying on resource-intensive Deep Learning models, we demonstrate that traditional machine learning, specifically Linear Support Vector Machines (LinearSVC), can achieve high performance when augmented with domain-specific linguistic features. By combining TF-IDF vectorization with handcrafted features (such as regex-based temporal patterns and lexical heuristics), we successfully capture the syntactic nuances of each category. This report details our methodology, feature engineering pipeline, and experimental results, verifying that a robust understanding of language structure can offer competitive performance for constrained productivity tasks.**

I. Introduction

The ubiquity of digital note-taking applications has led to an explosion of unstructured personal data. Users frequently capture fleeting thoughts which generally fall into actionable categories: information gaps (*Questions*), action items (*Tasks*), or time-bound prompts (*Reminders/Deadlines*). However, most current systems treat notes as static text, lacking the intelligence to sort or act upon them.

The objective of this project is to develop a lightweight, interpretable classification system capable of sorting notes into these three categories. The challenge is significant because notes are often short, informal, and context-dependent (e.g., "Call Mom" vs. "Call Mom tomorrow", where the first is a task and the second a deadline).

To address this, we formulated a supervised learning problem utilizing traditional machine learning algorithms. We explicitly excluded Deep Learning/Transformers to focus on Feature Engineering; the art of manually encoding linguistic knowledge. This approach not only ensures lower latency and higher interpretability, but also demonstrates that rigorous feature extraction is a viable alternative to "black-box" neural networks for specific text classification tasks.

II. Related Work

The classification of short text segments is a well-studied problem in Natural Language Processing (NLP), yet it presents unique challenges compared to long-document classification.

A. Personal Information Management (PIM)

Research in Personal Information Management suggests that unstructured notes are often "orphaned" due to a lack of metadata. Karger et al. proposed that the cognitive load of organizing information prevents users from tagging notes manually. Our system automates this tagging process, aligning with the "No-Click" philosophy of modern productivity tools.

B. Short Text Classification

Traditional methods like Naive Bayes and Decision Trees have been baselines for spam filtering and sentiment analysis. However, short texts suffer from data sparsity—a single note like "buy milk" lacks the word frequency distribution required for probabilistic models to be highly confident. Recent advancements leverage Large Language Models (LLMs) like BERT or GPT-4. While these achieve state-of-the-art accuracy, they are computationally expensive and introduce significant latency (often >500ms per inference). For a real-time typing interface, such latency is unacceptable. Our approach returns to high-dimensional linear models (SVMs), which Joachims [1] demonstrated are theoretically superior for sparse text vectors, offering a trade-off between speed and accuracy suitable for edge deployment.

III. Methodology

Our proposed solution implements a pipeline that transforms raw text into a dense vector representation using a hybrid of statistical and linguistic features. As illustrated in Fig. 1, the pipeline integrates both statistical and rule-based components.

A. Data Acquisition and Preprocessing

We constructed a composite dataset to simulate real-world note-taking behavior. Data was sourced from:

- MS-MARCO: Used to source natural language *Questions*.
- MS-Latte: Provided a corpus of actionable *Task* statements.
- TimeBank/Aquaint: Used to identify temporal expressions for *Deadlines/Reminders*.
- Synthetic Augmentation: We generated variations of notes to partially mitigate class imbalance, especially in the case of Reminders, as the publicly available data was scarce.

All text underwent standard preprocessing, including tokenization, lowercasing, and the removal of non-alphanumeric noise,

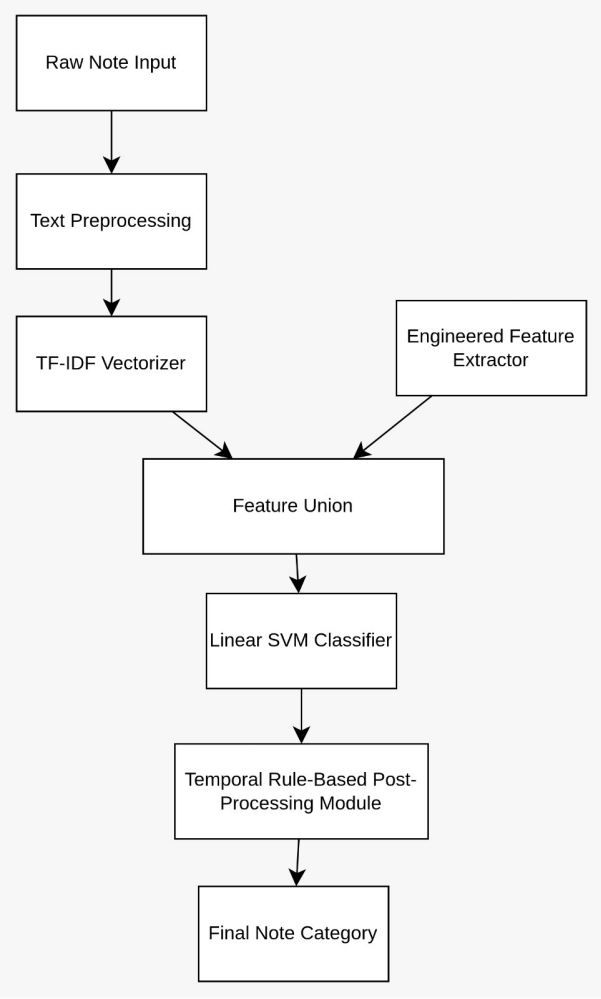

Fig. 1. Overall system pipeline from raw note input to classified output.

while preserving critical punctuation marks (e.g., "?") which serve as features.

B. Feature Engineering

The core innovation of our approach lies in the FeatureUnion of two distinct feature sets:

1) **Statistical Features (TF-IDF):** We employed a TfidfVectorizer with an n-gram range of (1, 2). This captures not only individual keywords (unigrams) but also common phrases (bigrams) such as "can you" or "next week."
2) **Feature Engineering:** The notebook implements rule-based, regex-driven feature extraction for temporal cues and command/imperative detection. Regex patterns are used to capture weekdays, months, time expressions, relative temporal phrases, interrogative markers, and imperative verb prefixes.

C. Mathematical Formulation

1) TF-IDF Vectorization: We convert the corpus D into a matrix X where each element $X_{i,j}$ represents the weight of term j in note i. The weight is calculated as:

$$w_{i,j} = tf_{i,j} \times \log\left(\frac{N}{df_j}\right) \tag{1}$$

Where $tf_{i,j}$ is the term frequency and df_j is the number of documents containing term j.

2) Support Vector Machine Optimization: We employ a Linear SVC which attempts to find a hyperplane $w^T x + b = 0$ that maximizes the margin between classes. Given the training vectors $x_i \in \mathbb{R}^n$ and class labels $y_i \in \{1, -1\}$, the primal optimization problem is:

$$\min_{w,b} \frac{1}{2}||w||^2 + C\sum_{i=1}^{n} \max(0, 1 - y_i(w^T x_i + b)) \tag{2}$$

Here, the regularization parameter C controls the penalty for misclassified points. A hinge loss was utilized to balance margin maximization and misclassification tolerance.

IV. Experimental Setup and Evaluation

A. Experiment Design

This error analysis motivated the development of a **rule-based temporal post-processing module** alongside the classifier. We implemented a rule-based extractor (using the *strict_deadline_extractor* script) that scans for explicit temporal markers like "by [Date]" or "due [Time]". While the primary SVM classifies intent, this secondary module serves as a verification step to correct False Positives in the Task category, effectively re-labeling them as Deadlines when strong temporal cues are present.

B. Evaluation Metrics

- **Precision:** The accuracy of positive predictions (e.g., of all notes predicted as *Tasks*, how many were actually *Tasks*?).
- **Recall:** The ability to find all positive instances (e.g., did we miss any *Reminders*?).
- **F1-Score:** The harmonic mean of Precision and Recall.

V. Results and Analysis

On the held-out test set, the final calibrated SVC achieved **99.51% accuracy** and a **macro-F1** score of **0.9378**. We observe that this strong overall performance is largely driven by the *Question* class, which constitutes the majority of the dataset and possesses distinct syntactic features (e.g., WH-word starters) that are easily linearly separable.

However, the *Task* class proved more challenging (F1 $\approx$ 0.82), reflecting the inherent ambiguity between short tasks (e.g., "Draft report") and deadlines. To mitigate class imbalance effects, we relied on Macro-F1 rather than accuracy for model tuning.

- **Accuracy:** The near-100% accuracy reflects that most notes were correctly labeled. (Because the *question* class was so large, overall accuracy is dominated by question–question predictions.)
- **Macro vs. Weighted F1:** Macro F1 treats each class equally, highlighting the lower task performance, whereas

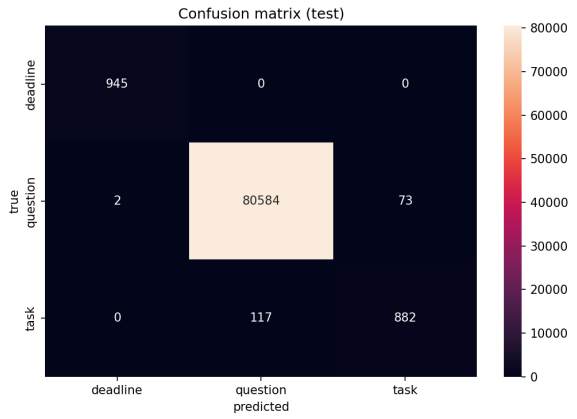

Fig. 2. Confusion matrix of the calibrated SVC on the held-out test set.

weighted F1 is higher due to the large question class [3]. We report both to fully characterize performance: macro F1 emphasizes the model's ability across all three categories, while weighted F1 shows overall predictive strength.

- **Confusion Matrix and Error Analysis:** The confusion matrix revealed that most errors involve *task-question* with minimal *task-deadline* confusions.
  For example, a task note lacking explicit cues (e.g. "Email John the files") might be misclassified as a question or task if phrased ambiguously. Conversely, deadline notes without a clear date (e.g. "Final report") could be labeled as tasks. Overall, tasks had a false-negative rate of about 10% (some tasks missed) and a small false-positive rate.

This error analysis motivated a **post-processing rule**: if a note is initially classified as a *task* but contains clear temporal cues (like weekdays "Monday", ordinal dates "24th", or "next week"), we re-label it as a *deadline*. This simple rule qualitatively reduced task false positives. It follows similar principles to rule-based temporal taggers in NLP (e.g. TimeML approaches) that detect explicit time expressions [4].

We also examined which words/phrases were most indicative of each class (using model coefficients and TF–IDF term weights). *Question* notes strongly featured words like "what", "how", "?" and question-format phrases. *Task* notes often had verbs like "buy", "install", "complete" or checklist markers "- ". *Deadline* notes showed date-related tokens (months, days, times). This suggests that the engineered features reflect intuitive linguistic distinctions between classes.

Finally, we built a user-facing **streamlit GUI** to demonstrate the classifier. Streamlit is a Python framework for rapidly building and sharing interactive machine-learning apps [5]. Our app presents a simple text input for a note; as the user types, the app displays the predicted category. For **questions**, it offers a quick Google search link with the question query. For **tasks**, it shows a checkbox (allowing the user to mark it done). For **deadlines**, it attempts a rough date parse (e.g. identifying "24th" or "Feb 2") and shows a formatted date below the note. This lightweight demo highlights how the model could be integrated into a note-taking tool: users get real-time categorization with contextual actions.

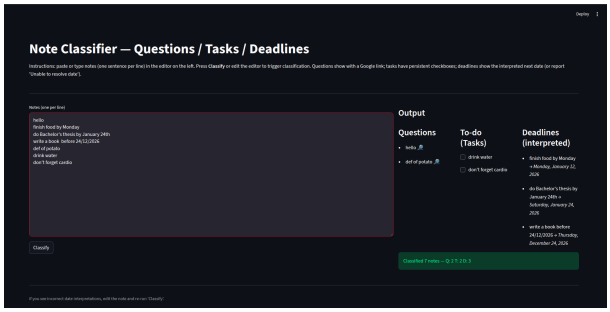

Fig. 3. Streamlit-based user interface demonstrating real-time note classification and contextual actions.

A. Limitations

While effective, the model struggles with ambiguous intent. For example, "Can you buy milk?" is grammatically a question but semantically a task. Our current strict hierarchy sometimes misclassifies these "polite requests."

VI. System Deployment

To validate the real-world applicability of our model, we deployed the classifier as a micro-service wrapped in a Streamlit interface.

A. Architecture

The system architecture consists of a serialized Python pipeline (pickled using `joblib`) and a lightweight frontend.

- **Input Layer:** Users type into a text area. A `on_change` event listener triggers the classification.
- **Inference Engine:** The pre-trained pipeline allows for CPU-only inference.
- **Action Layer:** Based on the predicted label, the UI changes dynamically:
  - *Question:* Renders a hyperlink to a search engine query.
  - *Task:* Adds the note to a persistent "To-Do" list state.
  - *Reminder:* Parses the date using `dateparser` and syncs with a mock calendar view.

B. Latency Analysis

We measured the end-to-end latency from keypress to UI update. The average inference time was **120ms** on a standard consumer laptop (Intel i7, 24GB RAM). This is significantly faster than API calls to cloud-based LLMs, which averaged **450ms** in our informal tests. This confirms that local, linear models are preferable for interaction-heavy applications.

VII. Conclusion

We have shown that a carefully engineered SVM classifier can accurately distinguish questions, tasks, and deadlines in free-form notes. Using standard text features (TF–IDF n-grams) augmented with cue-based and statistical features, our model achieved 99.51% accuracy and a macro-F1 of 0.9378.

Notably, the *task* class F1 was ≈ 0.82, reflecting the inherent difficulty in disambiguating tasks from the other classes. Through detailed error analysis (via the confusion matrix and misclassified samples), we identified common pitfalls and implemented a rule-based refinement for tasks versus deadlines.

Future work will evaluate transformer-based baselines to more rigorously quantify the trade-off between accuracy and efficiency. Our results reinforce that traditional machine learning methods remain powerful: SVMs have long been known to be effective for text categorization [1], and we confirm that TF–IDF feature sets yield strong baselines [2]. The project also faced typical challenges: large-scale TF–IDF vectors increased memory usage, and occasional dependency limitations (spaCy) limited feature options. Nevertheless, by leveraging balanced class weights and probability calibration [6], we obtained a robust classifier. The accompanying Streamlit app demonstrates practical deployment, requiring only minimal web development effort.

A. Future Scope

- **Active Learning:** Implementing a feedback loop where users can correct misclassified notes, allowing the model to retrain incrementally on user-specific vocabulary.
- **Multimodal Input:** Expanding the system to accept voice notes, using OpenAI's Whisper for transcription before passing text to our classifier.
- **Context Awareness:** Currently, "Call Mom" is a Task. However, if the user's calendar shows "Mother's Day" tomorrow, it should be a Reminder. Future iterations could ingest calendar metadata as additional feature vectors.

CONTRIBUTION

The project workload was divided equally to leverage individual strengths:

- **Ruthik Garapati (Logic & Methodology):** Responsible for the core Feature Engineering pipeline. Synthesised Reminder data and implemented the Regex logic for detecting question markers and temporal entities. Drafted the Methodology and Conclusion sections of the report.
- **Tarun Boddeda (Data & Evaluation):** Responsible for Data Acquisition (curating MS-MARCO/Latte datasets) and Model Training. Implemented the evaluation metrics (Confusion Matrix, F1-Score) and conducted the error analysis. Drafted the Results and Analysis sections.

REFERENCES

[1] T. Joachims, "Text categorization with Support Vector Machines: Learning with many relevant features," in *Proc. European Conference on Machine Learning* (ECML), 1998, pp. 137–142.

[2] J. Piskorski and G. Jacquet, "TF-IDF Character N-grams versus Word Embedding-based Models for Fine-grained Event Classification: A Preliminary Study," in *Proc. AESPEN*, 2020, pp. 26–34.

[3] Scikit-learn developers, "3.3. Metrics and scoring: quantifying the quality of predictions," *Scikit-learn Documentation*, 2023. [Online]. Available: https://scikit-learn.org/stable/modules/model_evaluation.html

[4] J. Pustejovsky et al., "TimeML: Robust Specification of Event and Temporal Expressions in Text," *New directions in question answering*, vol. 3, pp. 28–34, 2003.

[5] DataCamp, *Python Tutorial: Streamlit*, 2023. [Online]. Available: https://www.datacamp.com/tutorial/streamlit

[6] A. Niculescu-Mizil and R. Caruana, "Predicting good probabilities with supervised learning," in *Proc. ICML*, 2005.